



# Approximation Algorithms for Big Data

## Lecture 6: Kernel Density Estimation

Spring 2026

Instructor: Tal Wagner

# Administrative Announcements

# Administrative Announcements

- Today is a 1-hour class due to Student Day events
  - They will be underway when we leave class (campus impacts, gate closures etc.)

# Administrative Announcements

- Today is a 1-hour class due to Student Day events
  - They will be underway when we leave class (campus impacts, gate closures etc.)
- On July 9 (penultimate class) we plan a guest lecture by **Mobileye**
  - Details to come

# Administrative Announcements

- Today is a 1-hour class due to Student Day events
  - They will be underway when we leave class (campus impacts, gate closures etc.)
- On July 9 (penultimate class) we plan a guest lecture by **Mobileye**
  - Details to come
- Class mid-point survey will be published tonight
  - Please take a few minutes to fill it and share your feedback

# Administrative Announcements

- Today is a 1-hour class due to Student Day events
  - They will be underway when we leave class (campus impacts, gate closures etc.)
- On July 9 (penultimate class) we plan a guest lecture by **Mobileye**
  - Details to come
- Class mid-point survey will be published tonight
  - Please take a few minutes to fill it and share your feedback
- If you email me, make sure to use correct address email
  - [talwag@tauex.tau.ac.il](mailto:talwag@tauex.tau.ac.il)
  - Outlook auto-completes my name to a mailbox I don't own

# Plan

- Deeper dive into KDE approximations
- Opportunity to cover advanced sampling techniques:
  - Basic sampling (recap): uniform sampling, MoM
  - Advanced sampling techniques:
    - Importance sampling
    - LSH as a sampling technique
- Today's plan:
  - Review kernel approximation (random features, RFF, LSH)
  - KDE as a data structure problem (additive error, relative error)
  - Recap basic sampling (uniform sampling, MoM)
  - Importance sampling

# Recap

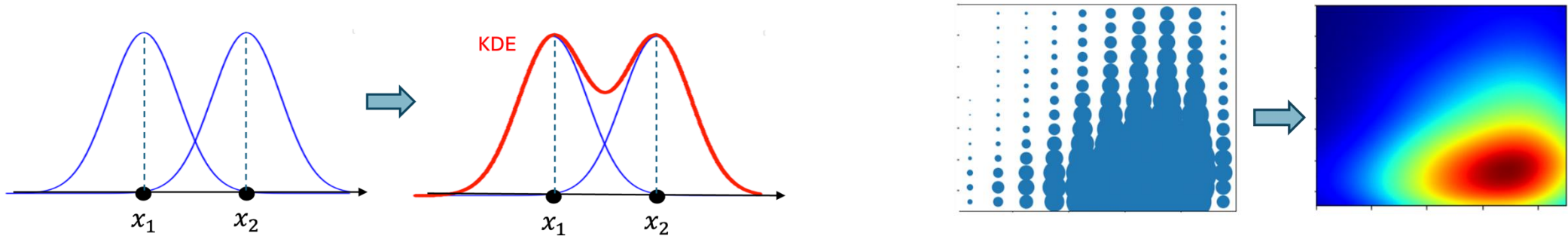
Kernels, KDE, LSH for kernels

# Kernels

- A **kernel** is a map  $k: \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$  that measures **similarity** between pairs of points
- Some commonly used kernels:
  - **Gaussian**:  $k(x, y) = e^{-\|x-y\|_2^2/2\sigma^2}$
  - **Laplacian**:  $k(x, y) = e^{-\|x-y\|_1/\sigma}$  or  $k(x, y) = e^{-\|x-y\|_2/\sigma}$
  - **Polynomial** (of degree  $p$ ):  $k(x, y) = \left(\frac{x^T y}{\sigma^2} + c\right)^p$
- $\sigma$  is the **bandwidth** parameter
  - Controls smoothness
  - We will often assume w.l.o.g. that  $\sigma = 1$  by scaling the points (but not always)

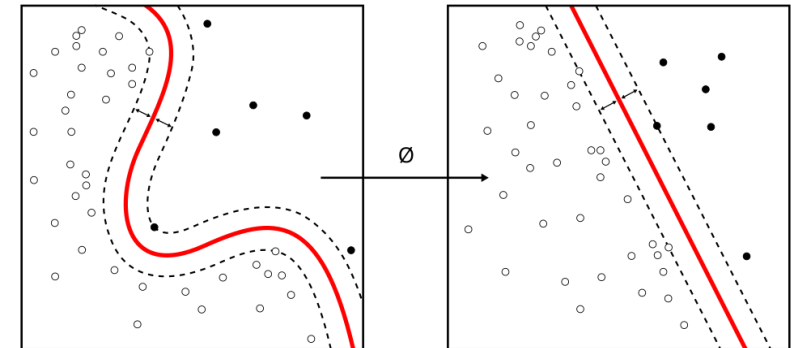
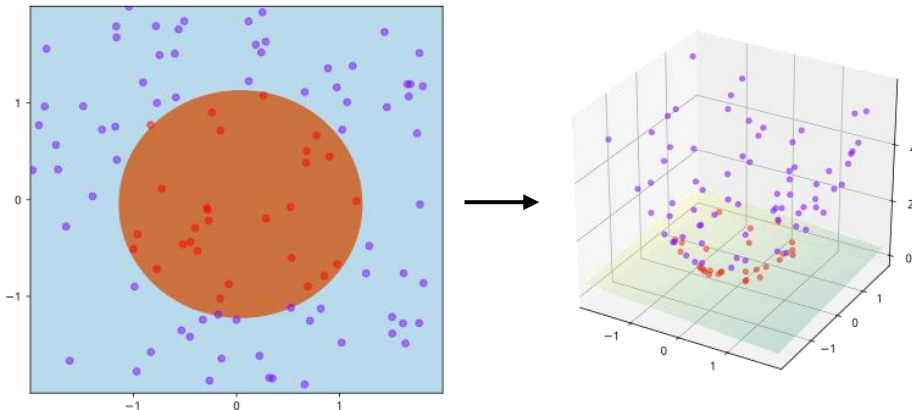
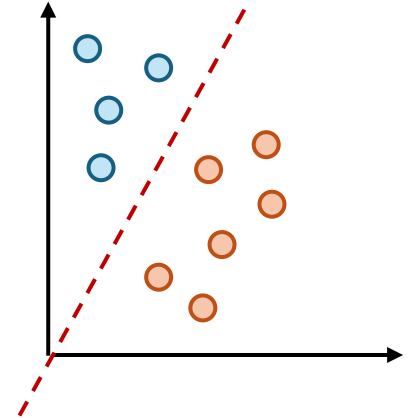
# Why Kernels? (1) Kernel Density Estimation

- Input: Set of  $n$  points  $X \subset \mathbb{R}^d$
- Task: Given a query  $y \in \mathbb{R}^d$ , return  $KDE_X(y) := \frac{1}{|X|} \sum_{x \in X} k(x, y)$
- Goal: Answer queries approximately with high probability



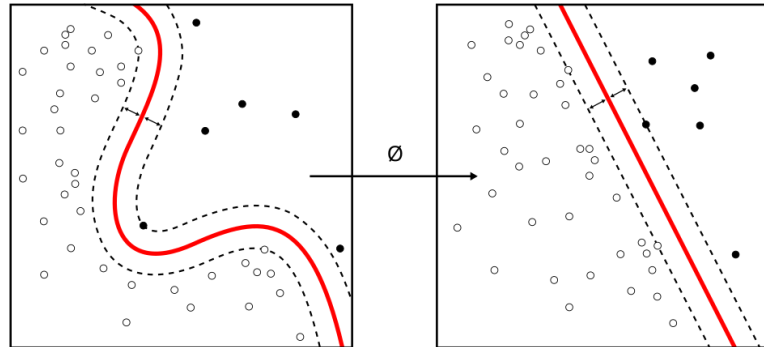
# Why Kernels? (2) Machine Learning

- **Support vector machines (SVMs)** are ML models for learning linear separators between labeled classes
  - Input: Dataset  $x_1, \dots, x_n \in \mathbb{R}^d$  with labels  $\ell_1, \dots, \ell_n \in \{1, -1\}$
  - Goal: Compute an optimal linear separator
- What if the dataset is not linearly separable?
- Approach: Map the dataset to a different space where it is!



# Kernels as Inner Products

- Intuition: kernels are inner products in a transformed space
- Meaning, there is a map  $\phi: \mathbb{R}^d \rightarrow \mathbb{R}^D$  such that  $k(x, y) = \phi(x)^T \phi(y)$



- However,  $D$  is typically **very large or infinite** and thus not useful algorithmically
  - Note that in kernel SVM we never needed to compute  $\phi$  or realize  $D$

# Kernel Approximation

- We saw two techniques – **what were they?**

- **Random features:** a distribution  $\mathcal{F}$  is a distribution over functions  $f: \mathbb{R}^d \rightarrow \mathbb{R}^m$  such that for all  $x, y \in \mathbb{R}^d$ :

$$k(x, y) = \mathbb{E}_{f \sim \mathcal{F}}[f(x)^T f(y)]$$

- **LSH:** a distribution  $\mathcal{H}$  over hash functions  $h$  such that for all  $x, y \in X$ :

$$k(x, y) = \Pr_{h \sim \mathcal{H}}[h(x) = h(y)]$$

- We saw **Random Fourier Features**
- We saw that **LSH for kernels** induces random features
- Today (and next time) we will see **a different way to use LSH for kernels**

# Random Fourier Features

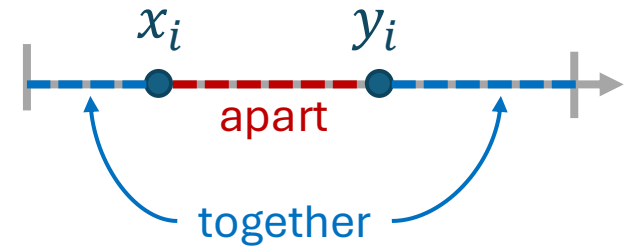
- Let  $\omega \in \mathbb{R}^m$  be a vector with iid Gaussian entries
- Let  $b \in [0, 2\pi]$  be uniformly random (debt from last time:  $b \in [0, \pi]$  is okay too)
- RFF for the Gaussian kernel  $k(x, y) = e^{-\|x-y\|_2^2/2}$  can be defined in one of two ways:

$$f_\omega(x) = (\sin(\omega^T x), \cos(\omega^T x)) \quad \text{Or} \quad f_{\omega,b}(x) = \sqrt{2} \cos(\omega^T x + b)$$

- In matrix form:

- Let  $W \in \mathbb{R}^{m \times d}$  be a matrix with iid Gaussian entries
- Let  $B \in \mathbb{R}^m$  a vector with iid entries uniform in  $[0, 2\pi]$
- Then:  $f_W(x) = \sin(Wx) \oplus \cos(Wx)$  Or  $f_{W,B}(x) = \sqrt{2} \cos(Wx + B)$

# LSH for the Laplacian Kernel



- Construction w saw in class
  1. Sample  $r \sim \text{Poisson}(d/\sigma)$ .
  2. Sample  $r$  i.i.d. axes  $i_1, \dots, i_r$  uniformly at random from  $\{1, \dots, d\}$ .
  3. Sample  $r$  i.i.d. thresholds  $\tau_1, \dots, \tau_r$  uniformly at random from  $[0,1]$ .
  4. The resulting hash function  $h: \mathbb{R}^d \rightarrow \{0,1\}^r$  is defined as the concatenation of thresholding bits:

$$h(x) = \mathbf{1}\{x_{i_1} > t_1\} \odot \mathbf{1}\{x_{i_2} > t_2\} \odot \dots \odot \mathbf{1}\{x_{i_r} > t_r\}$$

- Another construction: in HW2

# KDE Approximation by Random Features

# KDE Approximation by Random Features

- Recall **Hoeffding's inequality**: If  $X_1, \dots, X_r \in [a, b]$  are iid with mean  $\mathbb{E}[X_j] = \mu$ , then,

$$\Pr \left[ \left| \mu - \frac{1}{r} \sum_{j=1}^r X_j \right| \geq \varepsilon \right] \leq 2e^{-\frac{2r\varepsilon^2}{(b-a)^2}}$$

# KDE Approximation by Random Features

- Recall **Hoeffding's inequality**: If  $X_1, \dots, X_r \in [a, b]$  are iid with mean  $\mathbb{E}[X_j] = \mu$ , then,

$$\Pr \left[ \left| \mu - \frac{1}{r} \sum_{j=1}^r X_j \right| \geq \varepsilon \right] \leq 2e^{-\frac{2r\varepsilon^2}{(b-a)^2}}$$

- Suppose we have random features  $k(x, y) = \mathbb{E}_{f \sim \mathcal{F}}[f(x)^T f(y)]$

# KDE Approximation by Random Features

- Recall **Hoeffding's inequality**: If  $X_1, \dots, X_r \in [a, b]$  are iid with mean  $\mathbb{E}[X_j] = \mu$ , then,

$$\Pr \left[ \left| \mu - \frac{1}{r} \sum_{j=1}^r X_j \right| \geq \varepsilon \right] \leq 2e^{-\frac{2r\varepsilon^2}{(b-a)^2}}$$

- Suppose we have random features  $k(x, y) = \mathbb{E}_{f \sim \mathcal{F}}[f(x)^T f(y)]$
- Suppose also that for every  $f$  and every  $x, y \in \mathbb{R}^d$ , it holds that  $f(x)^T f(y) \in [a, b]$  for some constants  $a, b$  (both RFF and LSH satisfy this; with what constants  $a, b$ ?)

# KDE Approximation by Random Features

- Recall **Hoeffding's inequality**: If  $X_1, \dots, X_r \in [a, b]$  are iid with mean  $\mathbb{E}[X_j] = \mu$ , then,

$$\Pr \left[ \left| \mu - \frac{1}{r} \sum_{j=1}^r X_j \right| \geq \varepsilon \right] \leq 2e^{-\frac{2r\varepsilon^2}{(b-a)^2}}$$

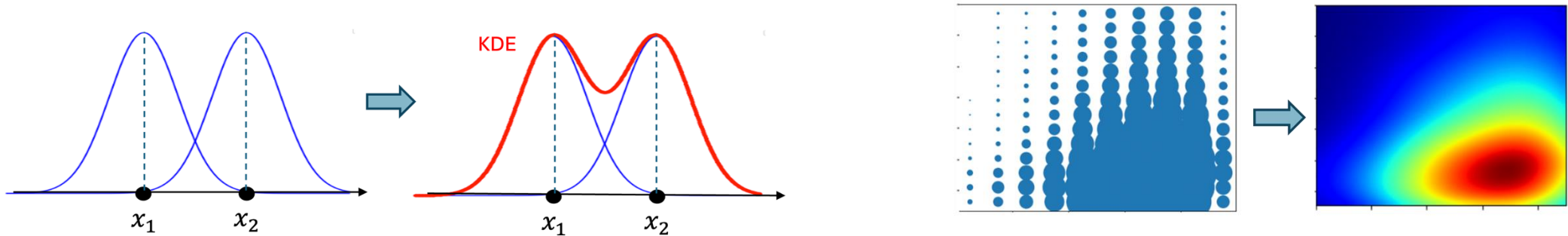
- Suppose we have random features  $k(x, y) = \mathbb{E}_{f \sim \mathcal{F}}[f(x)^T f(y)]$
- Suppose also that for every  $f$  and every  $x, y \in \mathbb{R}^d$ , it holds that  $f(x)^T f(y) \in [a, b]$  for some constants  $a, b$  (both RFF and LSH satisfy this; with what constants  $a, b$ ?)
- Then by Hoeffding's inequality: If draw  $f_1, \dots, f_r \sim \mathcal{F}$  iid with  $r = O(\log(\delta^{-1})/\varepsilon^2)$ ,

$$\forall x, y, \quad \Pr \left[ \left| k(x, y) - \frac{1}{r} \sum_{j=1}^r f_j(x)^T f_j(y) \right| \geq \varepsilon \right] \leq \delta$$

# Data Structures for Kernel Density Estimation

# Recap: Kernel Density Estimation (KDE)

- Input: Set of  $n$  points  $X \subset \mathbb{R}^d$
- Task: Given a query  $y \in \mathbb{R}^d$ , return  $KDE_X(y) := \frac{1}{|X|} \sum_{x \in X} k(x, y)$
- Goal: Answer queries approximately with high probability

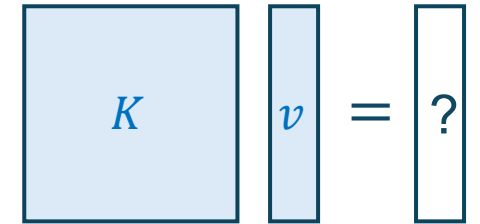


# Extra Motivation for KDE: Kernel Matrices

- Suppose we have a kernel  $k(x, y)$  and points  $x_1, \dots, x_n \in \mathbb{R}^d$
- Recall the **kernel matrix**:  $K \in \mathbb{R}^{n \times n}$  defined as  $K_{ij} = k(x_i, y_i)$

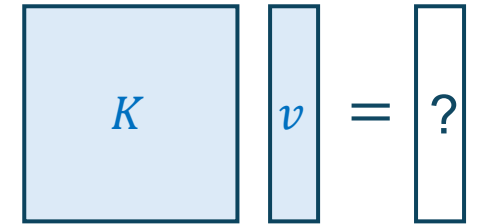
# Extra Motivation for KDE: Kernel Matrices

- Suppose we have a kernel  $k(x, y)$  and points  $x_1, \dots, x_n \in \mathbb{R}^d$
- Recall the **kernel matrix**:  $K \in \mathbb{R}^{n \times n}$  defined as  $K_{ij} = k(x_i, y_i)$
- Consider the kernel matrix-vector multiplication (KMVM) problem:
  - Given a vector  $v \in \mathbb{R}^d$ , return an approximation of  $Kv$


$$K v = ?$$

# Extra Motivation for KDE: Kernel Matrices

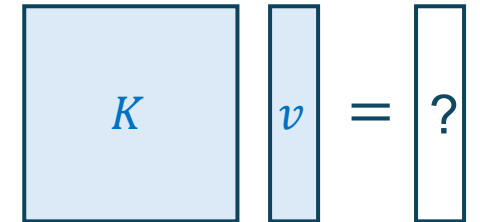
- Suppose we have a kernel  $k(x, y)$  and points  $x_1, \dots, x_n \in \mathbb{R}^d$
- Recall the **kernel matrix**:  $K \in \mathbb{R}^{n \times n}$  defined as  $K_{ij} = k(x_i, y_i)$
- Consider the kernel matrix-vector multiplication (KMVM) problem:
  - Given a vector  $v \in \mathbb{R}^d$ , return an approximation of  $Kv$
  - **Exact computation time?**



A diagram illustrating the kernel matrix-vector multiplication (KMVM) problem. It shows a light blue square matrix labeled  $K$  multiplied by a light blue vertical vector labeled  $v$ , resulting in a white vertical vector labeled with a question mark  $?$ . The equation is represented as  $K \cdot v = ?$ .

# Extra Motivation for KDE: Kernel Matrices

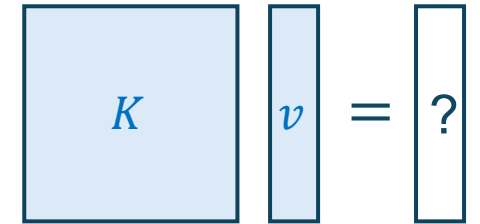
- Suppose we have a kernel  $k(x, y)$  and points  $x_1, \dots, x_n \in \mathbb{R}^d$
- Recall the **kernel matrix**:  $K \in \mathbb{R}^{n \times n}$  defined as  $K_{ij} = k(x_i, y_i)$
- Consider the kernel matrix-vector multiplication (KMVM) problem:
  - Given a vector  $v \in \mathbb{R}^d$ , return an approximation of  $Kv$ 
    - **Exact computation time?**
    - Since  $K$  has **specific structure** (a kernel matrix), can we do better?



A diagram illustrating the kernel matrix-vector multiplication (KMVM) problem. It shows a light blue square labeled  $K$  (representing the kernel matrix) multiplied by a light blue vertical rectangle labeled  $v$  (representing the vector). The result is shown as a white vertical rectangle with a question mark  $?$  inside, indicating the output of the multiplication.

# Extra Motivation for KDE: Kernel Matrices

- Suppose we have a kernel  $k(x, y)$  and points  $x_1, \dots, x_n \in \mathbb{R}^d$
- Recall the **kernel matrix**:  $K \in \mathbb{R}^{n \times n}$  defined as  $K_{ij} = k(x_i, y_i)$
- Consider the kernel matrix-vector multiplication (KMVM) problem:
  - Given a vector  $v \in \mathbb{R}^d$ , return an approximation of  $Kv$ 
    - **Exact computation time?**
    - Since  $K$  has **specific structure** (a kernel matrix), can we do better?


$$\begin{matrix} \boxed{K} \end{matrix} \begin{matrix} \boxed{v} \end{matrix} = \begin{matrix} \boxed{?} \end{matrix}$$

- **Observe:** 
$$(Kv)_j = \sum_{i=1}^n K_{ij} v_i = \sum_{i=1}^n v_i \cdot k(x_i, y)$$

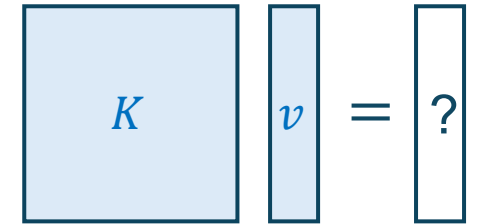
# Extra Motivation for KDE: Kernel Matrices

- Suppose we have a kernel  $k(x, y)$  and points  $x_1, \dots, x_n \in \mathbb{R}^d$
- Recall the **kernel matrix**:  $K \in \mathbb{R}^{n \times n}$  defined as  $K_{ij} = k(x_i, y_i)$
- Consider the kernel matrix-vector multiplication (KMVM) problem:
  - Given a vector  $v \in \mathbb{R}^d$ , return an approximation of  $Kv$ 
    - **Exact computation time?**
    - Since  $K$  has **specific structure** (a kernel matrix), can we do better?
- **Observe:** 
$$(Kv)_j = \sum_{i=1}^n K_{ij} v_i = \sum_{i=1}^n v_i \cdot k(x_i, y)$$
- This is a weighted variant of KDE – approximation techniques for KDE are useful

$$K v = ?$$

# Extra Motivation for KDE: Kernel Matrices

- Suppose we have a kernel  $k(x, y)$  and points  $x_1, \dots, x_n \in \mathbb{R}^d$
- Recall the **kernel matrix**:  $K \in \mathbb{R}^{n \times n}$  defined as  $K_{ij} = k(x_i, y_i)$
- Consider the kernel matrix-vector multiplication (KMVM) problem:
  - Given a vector  $v \in \mathbb{R}^d$ , return an approximation of  $Kv$ 
    - **Exact computation time?**
    - Since  $K$  has **specific structure** (a kernel matrix), can we do better?

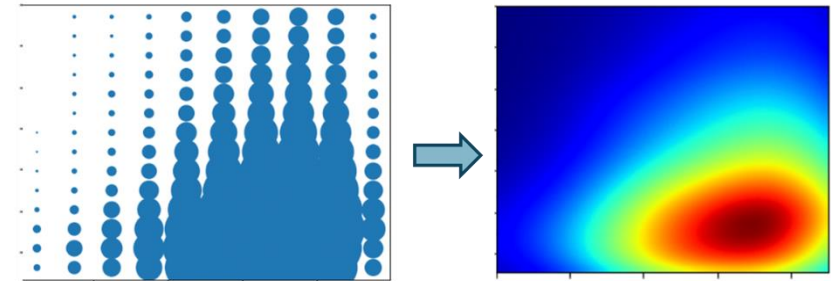

$$\begin{matrix} \boxed{K} \end{matrix} \begin{matrix} \boxed{v} \end{matrix} = \begin{matrix} \boxed{?} \end{matrix}$$

- **Observe:**  $(Kv)_j = \sum_{i=1}^n K_{ij} v_i = \sum_{i=1}^n v_i \cdot k(x_i, y)$
- This is a weighted variant of KDE – approximation techniques for KDE are useful
- KMVM arises often in machine learning, large language models, etc. (we'll see more)

# KDE Data Structures

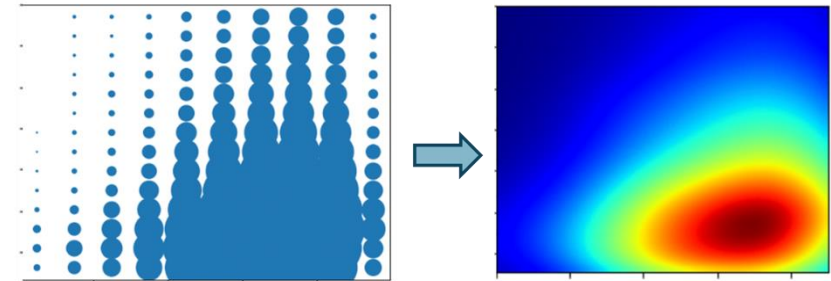
# KDE Data Structures

- A KDE data structure has two operations:
  - Build: Given  $n$  points  $X \subset \mathbb{R}^d$ , build data structure
  - Query: Given  $y \in \mathbb{R}^d$ , return an approximation  $\tilde{K}(y)$  of  $KDE_X(y)$
  - Goal: The  $\tilde{K}(y)$  needs to be a good approximation with high probability



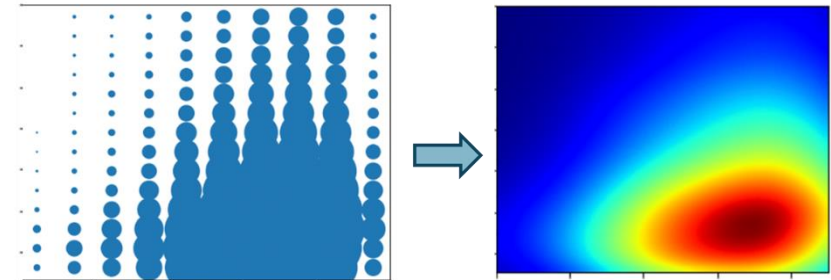
# KDE Data Structures

- A KDE data structure has two operations:
  - Build: Given  $n$  points  $X \subset \mathbb{R}^d$ , build data structure
  - Query: Given  $y \in \mathbb{R}^d$ , return an approximation  $\tilde{K}(y)$  of  $KDE_X(y)$
  - Goal: The  $\tilde{K}(y)$  needs to be a good approximation with high probability
- Computational cost measures?



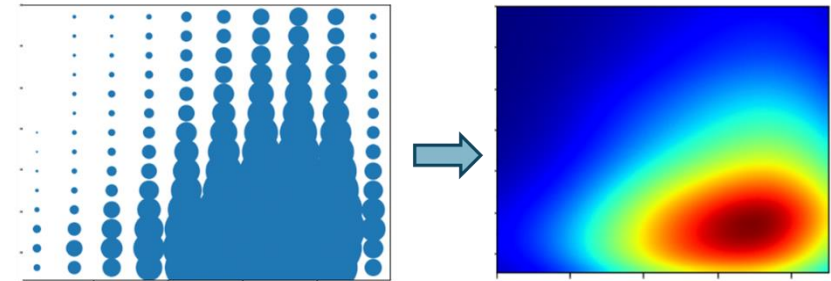
# KDE Data Structures

- A KDE data structure has two operations:
  - Build: Given  $n$  points  $X \subset \mathbb{R}^d$ , build data structure
  - Query: Given  $y \in \mathbb{R}^d$ , return an approximation  $\tilde{K}(y)$  of  $KDE_X(y)$
  - Goal: The  $\tilde{K}(y)$  needs to be a good approximation with high probability
- Computational cost measures?
  - Build time
  - Memory usage
  - Query time (main measure)



# KDE Data Structures

- A KDE data structure has two operations:
  - Build: Given  $n$  points  $X \subset \mathbb{R}^d$ , build data structure
  - Query: Given  $y \in \mathbb{R}^d$ , return an approximation  $\tilde{K}(y)$  of  $KDE_X(y)$
  - Goal: The  $\tilde{K}(y)$  needs to be a good approximation with high probability
- Computational cost measures?
  - Build time
  - Memory usage
  - Query time (main measure)
- Simplifying assumptions throughout:
  - $k(x, y)$  takes values in  $[0, 1]$
  - $k(x, y)$  takes time  $O(d)$  to compute



} Both holds for Gaussian, Laplacian, etc

# KDE with Additive Error: Random Features

- Suppose we have random features:  $k(x, y) = \mathbb{E}_{f \sim \mathcal{F}}[f(x)^T f(y)]$

# KDE with Additive Error: Random Features

- Suppose we have random features:  $k(x, y) = \mathbb{E}_{f \sim \mathcal{F}}[f(x)^T f(y)]$
- How can we get a KDE data structure?

# KDE with Additive Error: Random Features

- Suppose we have random features:  $k(x, y) = \mathbb{E}_{f \sim \mathcal{F}}[f(x)^T f(y)]$
- How can we get a KDE data structure?
  - Build:
    - Sample  $r$  random features  $f_1, \dots, f_r \sim \mathcal{F}$
    - For every  $f_j$ , compute and store  $F_j(X) := \frac{1}{|X|} \sum_{x \in X} f_j(x)$
  - Query: given  $y$ , return  $\tilde{K}(y) = \frac{1}{r} \sum_{j=1}^r F_j(X)^T f_j(y)$

# KDE with Additive Error: Random Features

- Suppose we have random features:  $k(x, y) = \mathbb{E}_{f \sim \mathcal{F}}[f(x)^T f(y)]$
- How can we get a KDE data structure?
  - Build:
    - Sample  $r$  random features  $f_1, \dots, f_r \sim \mathcal{F}$
    - For every  $f_j$ , compute and store  $F_j(X) := \frac{1}{|X|} \sum_{x \in X} f_j(x)$
  - Query: given  $y$ , return  $\tilde{K}(y) = \frac{1}{r} \sum_{j=1}^r F_j(X)^T f_j(y)$
- What are build time, memory, query time? For concreteness, analyze RFF

# KDE with Additive Error: Random Features

- Suppose we have random features:  $k(x, y) = \mathbb{E}_{f \sim \mathcal{F}}[f(x)^T f(y)]$
- How can we get a KDE data structure?
  - Build:
    - Sample  $r$  random features  $f_1, \dots, f_r \sim \mathcal{F}$
    - For every  $f_j$ , compute and store  $F_j(X) := \frac{1}{|X|} \sum_{x \in X} f_j(x)$
  - Query: given  $y$ , return  $\tilde{K}(y) = \frac{1}{r} \sum_{j=1}^r F_j(X)^T f_j(y)$
- What are build time, memory, query time? For concreteness, analyze RFF
- **Prove:** if  $\forall f, x, y$  we have  $|f(x)^T f(y)| = O(1)$ , and we take  $r = O(\log(\delta^{-1})/\varepsilon^2)$ ,

$$\forall x, y, \quad \Pr[|KDE_X(y) - \tilde{K}(y)| \geq \varepsilon] \leq \delta$$

# KDE with Additive Error: Random Sampling

# KDE with Additive Error: Random Sampling

- Consider the data structure:
  - Build: sample  $m$  points  $z_1, \dots, z_r$  iid uniformly at random from  $X$  (with replacement)
  - Query: given  $y$ , return  $\tilde{K}(y) = \frac{1}{r} \sum_{j=1}^r k(z_j, y)$

# KDE with Additive Error: Random Sampling

- Consider the data structure:
  - Build: sample  $m$  points  $z_1, \dots, z_r$  iid uniformly at random from  $X$  (with replacement)
  - Query: given  $y$ , return  $\tilde{K}(y) = \frac{1}{r} \sum_{j=1}^r k(z_j, y)$
- Build time, memory, query time?

# KDE with Additive Error: Random Sampling

- Consider the data structure:
  - Build: sample  $m$  points  $z_1, \dots, z_r$  iid uniformly at random from  $X$  (with replacement)
  - Query: given  $y$ , return  $\tilde{K}(y) = \frac{1}{r} \sum_{j=1}^r k(z_j, y)$
- Build time, memory, query time?
- **Claim:** With sample size  $r = O(\log(\delta^{-1})/\varepsilon^2)$ , this is a KDE data structure with additive error  $\varepsilon$  and failure probability  $\delta$ 
  - Proof: HW2

# KDE with Additive Error: Random Sampling

- Consider the data structure:
  - Build: sample  $m$  points  $z_1, \dots, z_r$  iid uniformly at random from  $X$  (with replacement)
  - Query: given  $y$ , return  $\tilde{K}(y) = \frac{1}{r} \sum_{j=1}^r k(z_j, y)$
- Build time, memory, query time?
- **Claim:** With sample size  $r = O(\log(\delta^{-1})/\varepsilon^2)$ , this is a KDE data structure with additive error  $\varepsilon$  and failure probability  $\delta$ 
  - Proof: HW2
- Remark: This alludes to [coresets](#), a much broader topic in KDE and in general

# KDE with Relative Error

# KDE with Relative Error

- **Recall:** A KDE data structure has two operations:
  - Build: Given  $n$  points  $X \subset \mathbb{R}^d$ , build data structure
  - Query: Given  $y \in \mathbb{R}^d$ , return an approximation  $\tilde{K}(y)$  of  $KDE_X(y)$
- **Recall:** Additive error guarantee: Goal is  $\tilde{K}(y) = KDE_X(y) \pm \varepsilon$  w.p.  $1 - \delta$

# KDE with Relative Error

- **Recall:** A KDE data structure has two operations:
  - Build: Given  $n$  points  $X \subset \mathbb{R}^d$ , build data structure
  - Query: Given  $y \in \mathbb{R}^d$ , return an approximation  $\tilde{K}(y)$  of  $KDE_X(y)$
- **Recall:** Additive error guarantee: Goal is  $\tilde{K}(y) = KDE_X(y) \pm \varepsilon$  w.p.  $1 - \delta$
- **Now:** Relative error guarantee
  - Let  $\varepsilon, \delta, \tau \in (0,1)$
  - Promise: the query  $y$  satisfies  $KDE_X(y) \geq \tau$
  - Goal:  $\tilde{K}(y) = (1 \pm \varepsilon) \cdot KDE_X(y)$  w.p.  $1 - \delta$

# KDE with Relative Error

- **Recall:** A KDE data structure has two operations:
  - Build: Given  $n$  points  $X \subset \mathbb{R}^d$ , build data structure
  - Query: Given  $y \in \mathbb{R}^d$ , return an approximation  $\tilde{K}(y)$  of  $KDE_X(y)$
- **Recall:** Additive error guarantee: Goal is  $\tilde{K}(y) = KDE_X(y) \pm \varepsilon$  w.p.  $1 - \delta$
- **Now:** Relative error guarantee
  - Let  $\varepsilon, \delta, \tau \in (0,1)$
  - Promise: the query  $y$  satisfies  $KDE_X(y) \geq \tau$
  - Goal:  $\tilde{K}(y) = (1 \pm \varepsilon) \cdot KDE_X(y)$  w.p.  $1 - \delta$
- **Why is this better than additive error?** Consider  $\tau \ll \varepsilon$

# KDE with Relative Error: Vanilla Solution

# KDE with Relative Error: Vanilla Solution

- KDE with **additive** error:  $\tilde{K}(y) = KDE_X(y) \pm \varepsilon$  w.p.  $1 - \delta$
- KDE with **relative** error: if  $KDE_X(y) \geq \tau$ , then  $\tilde{K}(y) = (1 \pm \varepsilon) \cdot KDE_X(y)$  w.p.  $1 - \delta$

# KDE with Relative Error: Vanilla Solution

- KDE with **additive** error:  $\tilde{K}(y) = KDE_X(y) \pm \varepsilon$  w.p.  $1 - \delta$
- KDE with **relative** error: if  $KDE_X(y) \geq \tau$ , then  $\tilde{K}(y) = (1 \pm \varepsilon) \cdot KDE_X(y)$  w.p.  $1 - \delta$
- How we trivially turn KDE with additive error into KDE with relative error?

# KDE with Relative Error: Vanilla Solution

- KDE with **additive** error:  $\tilde{K}(y) = KDE_X(y) \pm \varepsilon$  w.p.  $1 - \delta$
- KDE with **relative** error: if  $KDE_X(y) \geq \tau$ , then  $\tilde{K}(y) = (1 \pm \varepsilon) \cdot KDE_X(y)$  w.p.  $1 - \delta$
- How we trivially turn KDE with additive error into KDE with relative error?
- What query time do we get from Random Features or Random Sampling?

# KDE with Relative Error: Vanilla Solution

- KDE with **additive** error:  $\tilde{K}(y) = KDE_X(y) \pm \varepsilon$  w.p.  $1 - \delta$
- KDE with **relative** error: if  $KDE_X(y) \geq \tau$ , then  $\tilde{K}(y) = (1 \pm \varepsilon) \cdot KDE_X(y)$  w.p.  $1 - \delta$
- How we trivially turn KDE with additive error into KDE with relative error?
- What query time do we get from Random Features or Random Sampling?
  - $O\left(d \cdot \frac{\log(\delta^{-1})}{\tau^2 \varepsilon^2}\right)$

# KDE with Relative Error: Vanilla Solution

- KDE with **additive** error:  $\tilde{K}(y) = KDE_X(y) \pm \varepsilon$  w.p.  $1 - \delta$
- KDE with **relative** error: if  $KDE_X(y) \geq \tau$ , then  $\tilde{K}(y) = (1 \pm \varepsilon) \cdot KDE_X(y)$  w.p.  $1 - \delta$
- How we trivially turn KDE with additive error into KDE with relative error?
- What query time do we get from Random Features or Random Sampling?
  - $O\left(d \cdot \frac{\log(\delta^{-1})}{\tau^2 \varepsilon^2}\right)$
  - This is a **suboptimal** analysis of Random Sampling – we'll see  $O\left(d \cdot \frac{\log(\delta^{-1})}{\tau \varepsilon^2}\right)$

# Recap: Median of Means

# Recap: Median of Means

- Suppose we have a random variable  $Z$  with  $\mathbb{E}[Z] = \mu$  and variance  $\text{Var}(Z)$ 
  - For our current discussion,  $Z$  is an unbiased KDE estimator:  $\mu = \text{KDE}_x(y)$

# Recap: Median of Means

- Suppose we have a random variable  $Z$  with  $\mathbb{E}[Z] = \mu$  and variance  $\text{Var}(Z)$ 
  - For our current discussion,  $Z$  is an **unbiased KDE estimator**:  $\mu = \text{KDE}_X(y)$
- **Theorem (MoM)**: with  $r = O\left(\frac{\text{Var}(Z)}{\mu^2 \varepsilon^2} \log\left(\frac{1}{\delta}\right)\right)$  samples, we can get an estimate  $\tilde{Z}$  that satisfies  $\tilde{Z} = (1 \pm \varepsilon)\mu$  w.p.  $1 - \delta$  (**Proof?**)

# Recap: Median of Means

- Suppose we have a random variable  $Z$  with  $\mathbb{E}[Z] = \mu$  and variance  $\text{Var}(Z)$ 
  - For our current discussion,  $Z$  is an **unbiased KDE estimator**:  $\mu = \text{KDE}_X(y)$
- **Theorem (MoM)**: with  $r = O\left(\frac{\text{Var}(Z)}{\mu^2 \varepsilon^2} \log\left(\frac{1}{\delta}\right)\right)$  samples, we can get an estimate  $\tilde{Z}$  that satisfies  $\tilde{Z} = (1 \pm \varepsilon)\mu$  w.p.  $1 - \delta$  (**Proof?**)
  - Let  $\bar{Z}$  be the average of  $\bar{r} = O\left(\frac{\text{Var}(Z)}{\mu^2 \varepsilon^2}\right)$  iid samples of  $Z$ .
  - By Chebyshev's inequality,  $\bar{Z} = (1 \pm \varepsilon)\mu$  w.p.  $2/3$ .

# Recap: Median of Means

- Suppose we have a random variable  $Z$  with  $\mathbb{E}[Z] = \mu$  and variance  $\text{Var}(Z)$ 
  - For our current discussion,  $Z$  is an **unbiased KDE estimator**:  $\mu = \text{KDE}_X(y)$
- **Theorem (MoM)**: with  $r = O\left(\frac{\text{Var}(Z)}{\mu^2 \varepsilon^2} \log\left(\frac{1}{\delta}\right)\right)$  samples, we can get an estimate  $\tilde{Z}$  that satisfies  $\tilde{Z} = (1 \pm \varepsilon)\mu$  w.p.  $1 - \delta$  (**Proof?**)
  - Let  $\bar{Z}$  be the average of  $\bar{r} = O\left(\frac{\text{Var}(Z)}{\mu^2 \varepsilon^2}\right)$  iid samples of  $Z$ .
  - By Chebyshev's inequality,  $\bar{Z} = (1 \pm \varepsilon)\mu$  w.p.  $2/3$ .
  - Let  $\tilde{Z}$  be the median of  $O(\log(1/\delta))$  repetitions of  $\bar{Z}$ .
  - By Chernoff's inequality,  $\tilde{Z} = (1 \pm \varepsilon)\mu$  w.p.  $1 - \delta$ .

# Recap: Median of Means

- Suppose we have a random variable  $Z$  with  $\mathbb{E}[Z] = \mu$  and variance  $\text{Var}(Z)$ 
  - For our current discussion,  $Z$  is an **unbiased KDE estimator**:  $\mu = \text{KDE}_X(y)$
- **Theorem (MoM)**: with  $r = O\left(\frac{\text{Var}(Z)}{\mu^2 \varepsilon^2} \log\left(\frac{1}{\delta}\right)\right)$  samples, we can get an estimate  $\tilde{Z}$  that satisfies  $\tilde{Z} = (1 \pm \varepsilon)\mu$  w.p.  $1 - \delta$  (**Proof?**)
  - Let  $\bar{Z}$  be the average of  $\bar{r} = O\left(\frac{\text{Var}(Z)}{\mu^2 \varepsilon^2}\right)$  iid samples of  $Z$ .
  - By Chebyshev's inequality,  $\bar{Z} = (1 \pm \varepsilon)\mu$  w.p.  $2/3$ .
  - Let  $\tilde{Z}$  be the median of  $O(\log(1/\delta))$  repetitions of  $\bar{Z}$ .
  - By Chernoff's inequality,  $\tilde{Z} = (1 \pm \varepsilon)\mu$  w.p.  $1 - \delta$ .
- For simplicity, **we will suppress  $\delta$  from now on**

# KDE with Relative Error by Random Sampling

- For brevity we will denote  $\mu = KDE_X(y)$

# KDE with Relative Error by Random Sampling

- For brevity we will denote  $\mu = KDE_X(y)$
- Random Sampling KDE estimator:
  - Sample  $z$  from  $X$  uniformly at random
  - The estimator is:  $Z := k(z, y)$

# KDE with Relative Error by Random Sampling

- For brevity we will denote  $\mu = KDE_X(y)$
- Random Sampling KDE estimator:
  - Sample  $z$  from  $X$  uniformly at random
  - The estimator is:  $Z := k(z, y)$
- Expectation:  $\mathbb{E}[Z] =$  ???

# KDE with Relative Error by Random Sampling

- For brevity we will denote  $\mu = KDE_X(y)$
- Random Sampling KDE estimator:
  - Sample  $z$  from  $X$  uniformly at random
  - The estimator is:  $Z := k(z, y)$
- Expectation:  $\mathbb{E}[Z] = \sum_{x \in X} \frac{1}{|X|} k(x, y) = KDE_X(y)$

# KDE with Relative Error by Random Sampling

- For brevity we will denote  $\mu = KDE_X(y)$
- Random Sampling KDE estimator:
  - Sample  $z$  from  $X$  uniformly at random
  - The estimator is:  $Z := k(z, y)$
- Expectation:  $\mathbb{E}[Z] = \sum_{x \in X} \frac{1}{|X|} k(x, y) = KDE_X(y)$
- Variance:  $\text{Var}(Z) = \mathbb{E}[Z^2] - (\mathbb{E}[Z])^2 \leq \mathbb{E}[Z^2]$

# KDE with Relative Error by Random Sampling

- For brevity we will denote  $\mu = KDE_X(y)$
- Random Sampling KDE estimator:
  - Sample  $z$  from  $X$  uniformly at random
  - The estimator is:  $Z := k(z, y)$
- Expectation:  $\mathbb{E}[Z] = \sum_{x \in X} \frac{1}{|X|} k(x, y) = KDE_X(y)$
- Variance:  $\text{Var}(Z) = \mathbb{E}[Z^2] - (\mathbb{E}[Z])^2 \leq \mathbb{E}[Z^2]$
- $$= \sum_{x \in X} \frac{1}{|X|} k(x, y)^2$$

# KDE with Relative Error by Random Sampling

- For brevity we will denote  $\mu = KDE_X(y)$
- Random Sampling KDE estimator:
  - Sample  $z$  from  $X$  uniformly at random
  - The estimator is:  $Z := k(z, y)$
- Expectation:  $\mathbb{E}[Z] = \sum_{x \in X} \frac{1}{|X|} k(x, y) = KDE_X(y)$
- Variance:  $\text{Var}(Z) = \mathbb{E}[Z^2] - (\mathbb{E}[Z])^2 \leq \mathbb{E}[Z^2]$ 
  - $$= \sum_{x \in X} \frac{1}{|X|} k(x, y)^2$$
  - $$\leq \max_{x^* \in X} k(x^*, y) \sum_{x \in X} \frac{1}{|X|} k(x, y)$$

# KDE with Relative Error by Random Sampling

- For brevity we will denote  $\mu = KDE_X(y)$
- Random Sampling KDE estimator:
  - Sample  $z$  from  $X$  uniformly at random
  - The estimator is:  $Z := k(z, y)$
- Expectation:  $\mathbb{E}[Z] = \sum_{x \in X} \frac{1}{|X|} k(x, y) = KDE_X(y)$
- Variance:  $\text{Var}(Z) = \mathbb{E}[Z^2] - (\mathbb{E}[Z])^2 \leq \mathbb{E}[Z^2]$ 
  - $= \sum_{x \in X} \frac{1}{|X|} k(x, y)^2$
  - $\leq \max_{x^* \in X} k(x^*, y) \sum_{x \in X} \frac{1}{|X|} k(x, y)$
  - $\leq KDE_X(y)$

# KDE with Relative Error by Random Sampling

- For brevity we will denote  $\mu = KDE_X(y)$
- **Random Sampling KDE estimator:**
  - Sample  $z$  from  $X$  uniformly at random
  - The estimator is:  $Z := k(z, y)$
- **Expectation:**  $\mathbb{E}[Z] = \sum_{x \in X} \frac{1}{|X|} k(x, y) = KDE_X(y)$
- **Variance:**  $\text{Var}(Z) = \mathbb{E}[Z^2] - (\mathbb{E}[Z])^2 \leq \mathbb{E}[Z^2]$ 
  - $= \sum_{x \in X} \frac{1}{|X|} k(x, y)^2$
  - $\leq \max_{x^* \in X} k(x^*, y) \sum_{x \in X} \frac{1}{|X|} k(x, y)$
  - $\leq KDE_X(y)$

- We got:

$$\mathbb{E}[Z] = \mu \text{ and } \text{Var}(Z) \leq \mu$$

# KDE with Relative Error by Random Sampling

- For brevity we will denote  $\mu = KDE_X(y)$
- **Random Sampling KDE estimator:**
  - Sample  $z$  from  $X$  uniformly at random
  - The estimator is:  $Z := k(z, y)$
- **Expectation:**  $\mathbb{E}[Z] = \sum_{x \in X} \frac{1}{|X|} k(x, y) = KDE_X(y)$
- **Variance:**  $\text{Var}(Z) = \mathbb{E}[Z^2] - (\mathbb{E}[Z])^2 \leq \mathbb{E}[Z^2]$ 
  - $= \sum_{x \in X} \frac{1}{|X|} k(x, y)^2$
  - $\leq \max_{x^* \in X} k(x^*, y) \sum_{x \in X} \frac{1}{|X|} k(x, y)$
  - $\leq KDE_X(y)$

- We got:

$$\mathbb{E}[Z] = \mu \text{ and } \text{Var}(Z) \leq \mu$$

- By MoM:  $O\left(\frac{\text{Var}(Z)}{\mu^2 \varepsilon^2}\right) = O\left(\frac{1}{\mu \varepsilon^2}\right)$   
samples suffice

# KDE with Relative Error by Random Sampling

- For brevity we will denote  $\mu = KDE_X(y)$
- **Random Sampling KDE estimator:**
  - Sample  $z$  from  $X$  uniformly at random
  - The estimator is:  $Z := k(z, y)$
- **Expectation:**  $\mathbb{E}[Z] = \sum_{x \in X} \frac{1}{|X|} k(x, y) = KDE_X(y)$
- **Variance:**  $\text{Var}(Z) = \mathbb{E}[Z^2] - (\mathbb{E}[Z])^2 \leq \mathbb{E}[Z^2]$ 
  - $= \sum_{x \in X} \frac{1}{|X|} k(x, y)^2$
  - $\leq \max_{x^* \in X} k(x^*, y) \sum_{x \in X} \frac{1}{|X|} k(x, y)$
  - $\leq KDE_X(y)$

- We got:

$$\mathbb{E}[Z] = \mu \text{ and } \text{Var}(Z) \leq \mu$$

- By MoM:  $O\left(\frac{\text{Var}(Z)}{\mu^2 \varepsilon^2}\right) = O\left(\frac{1}{\mu \varepsilon^2}\right)$  samples suffice
- Since  $\mu \geq \tau$ ,  $O\left(\frac{1}{\tau \varepsilon^2}\right)$  samples suffice.



# KDE with Relative Error by Random Sampling

- For brevity we will denote  $\mu = KDE_X(y)$
- **Random Sampling KDE estimator:**
  - Sample  $z$  from  $X$  uniformly at random
  - The estimator is:  $Z := k(z, y)$
- **Expectation:**  $\mathbb{E}[Z] = \sum_{x \in X} \frac{1}{|X|} k(x, y) = KDE_X(y)$
- **Variance:**  $\text{Var}(Z) = \mathbb{E}[Z^2] - (\mathbb{E}[Z])^2 \leq \mathbb{E}[Z^2]$ 
  - $= \sum_{x \in X} \frac{1}{|X|} k(x, y)^2$
  - $\leq \max_{x^* \in X} k(x^*, y) \sum_{x \in X} \frac{1}{|X|} k(x, y)$
  - $\leq KDE_X(y)$

- We got:

$$\mathbb{E}[Z] = \mu \text{ and } \text{Var}(Z) \leq \mu$$

- By MoM:  $O\left(\frac{\text{Var}(Z)}{\mu^2 \varepsilon^2}\right) = O\left(\frac{1}{\mu \varepsilon^2}\right)$  samples suffice
  - Since  $\mu \geq \tau$ ,  $O\left(\frac{1}{\tau \varepsilon^2}\right)$  samples suffice.
- 

- Remark: We don't need the median with Random Sampling, it's enough to take the average of all samples.

# Reducing the Sample Size

# Reducing the Sample Size

- Current status:
  - We have a generic estimator framework for KDE approximation
  - Sample size is governed by the variance of the estimator
  - We saw that uniform random sampling requires  $O\left(\frac{1}{\tau \varepsilon^2}\right)$  samples

# Reducing the Sample Size

- Current status:
  - We have a generic estimator framework for KDE approximation
  - Sample size is governed by the variance of the estimator
  - We saw that uniform random sampling requires  $O\left(\frac{1}{\tau \varepsilon^2}\right)$  samples
  - What could we improve here?

# Reducing the Sample Size

- Current status:
  - We have a generic estimator framework for KDE approximation
  - Sample size is governed by the variance of the estimator
  - We saw that uniform random sampling requires  $O\left(\frac{1}{\tau \varepsilon^2}\right)$  samples
  - What could we improve here?
- Goal: Improve the dependence on  $\tau$  to  $\left(\frac{1}{\tau^\alpha \varepsilon^2}\right)$  for some  $\alpha \in (0,1)$

# Reducing the Sample Size

- Current status:
  - We have a generic estimator framework for KDE approximation
  - Sample size is governed by the variance of the estimator
  - We saw that uniform random sampling requires  $O\left(\frac{1}{\tau \varepsilon^2}\right)$  samples
  - What could we improve here?
- Goal: Improve the dependence on  $\tau$  to  $\left(\frac{1}{\tau^\alpha \varepsilon^2}\right)$  for some  $\alpha \in (0,1)$
- How do we get better sample size, i.e., how can we reduce the variance?

# Importance Sampling

# Importance Sampling

- Instead of sampling point from  $X$  with equal probabilities, we assign each point **its own probability** to be included in the sample

# Importance Sampling

- Instead of sampling point from  $X$  with equal probabilities, we assign each point **its own probability** to be included in the sample
- Intuitively: some points are more important than others for estimating the KDE (**which ones?**), hence we want to sample according to their importance

# Importance Sampling

- Instead of sampling point from  $X$  with equal probabilities, we assign each point **its own probability** to be included in the sample
- Intuitively: some points are more important than others for estimating the KDE (**which ones?**), hence we want to sample according to their importance
- **Formally:**
  - Let  $x_1, \dots, x_n$  be the points in  $X$
  - Let  $p_1, \dots, p_n \in [0,1]$  be corresponding probabilities such that  $\sum_{i=1}^n p_i = 1$

# Importance Sampling

- Instead of sampling point from  $X$  with equal probabilities, we assign each point **its own probability** to be included in the sample
- Intuitively: some points are more important than others for estimating the KDE (**which ones?**), hence we want to sample according to their importance
- **Formally:**
  - Let  $x_1, \dots, x_n$  be the points in  $X$
  - Let  $p_1, \dots, p_n \in [0,1]$  be corresponding probabilities such that  $\sum_{i=1}^n p_i = 1$
  - **Importance sampling estimator:**
    - Sample  $x_i$  from  $X$  with probability  $p_i$
    - Estimator:  $Z := \frac{1}{n \cdot p_i} k(x_i, y)$

# First and Second Moments

- $\mathbb{E}[Z] =$

# First and Second Moments

- $\mathbb{E}[Z] = \sum_{i=1}^n p_i \cdot \frac{1}{n \cdot p_i} k(x_i, y) = KDE_X(y)$

# First and Second Moments

- $\mathbb{E}[Z] = \sum_{i=1}^n p_i \cdot \frac{1}{n \cdot p_i} k(x_i, y) = KDE_X(y)$

- $\mathbb{E}[Z^2] =$

???

# First and Second Moments

- $\mathbb{E}[Z] = \sum_{i=1}^n p_i \cdot \frac{1}{n \cdot p_i} k(x_i, y) = KDE_X(y)$
- $\mathbb{E}[Z^2] = \sum_{i=1}^n p_i \cdot \left( \frac{1}{n \cdot p_i} k(x_i, y) \right)^2 = \frac{1}{n} \sum_{i=1}^n \frac{k(x_i, y)^2}{p_i}$

# First and Second Moments

- $\mathbb{E}[Z] = \sum_{i=1}^n p_i \cdot \frac{1}{n \cdot p_i} k(x_i, y) = KDE_X(y)$
- $\mathbb{E}[Z^2] = \sum_{i=1}^n p_i \cdot \left( \frac{1}{n \cdot p_i} k(x_i, y) \right)^2 = \frac{1}{n} \sum_{i=1}^n \frac{k(x_i, y)^2}{p_i}$
- What choice of  $\{p_i\}$  minimizes the second moment / variance?

# First and Second Moments

- $\mathbb{E}[Z] = \sum_{i=1}^n p_i \cdot \frac{1}{n \cdot p_i} k(x_i, y) = KDE_X(y)$
- $\mathbb{E}[Z^2] = \sum_{i=1}^n p_i \cdot \left( \frac{1}{n \cdot p_i} k(x_i, y) \right)^2 = \frac{1}{n} \sum_{i=1}^n \frac{k(x_i, y)^2}{p_i}$
- What choice of  $\{p_i\}$  minimizes the second moment / variance?
  - Denote  $k_i = k(x_i, y)$  for brevity
  - We need to solve the optimization problem:

$$\min_{p_1, \dots, p_n} \sum_{i=1}^n \frac{k_i^2}{p_i} \quad \text{subject to} \quad \sum_{i=1}^n p_i = 1$$

# First and Second Moments

- $\mathbb{E}[Z] = \sum_{i=1}^n p_i \cdot \frac{1}{n \cdot p_i} k(x_i, y) = KDE_X(y)$
- $\mathbb{E}[Z^2] = \sum_{i=1}^n p_i \cdot \left( \frac{1}{n \cdot p_i} k(x_i, y) \right)^2 = \frac{1}{n} \sum_{i=1}^n \frac{k(x_i, y)^2}{p_i}$
- What choice of  $\{p_i\}$  minimizes the second moment / variance?
  - Denote  $k_i = k(x_i, y)$  for brevity
  - We need to solve the optimization problem:
$$\min_{p_1, \dots, p_n} \sum_{i=1}^n \frac{k_i^2}{p_i} \quad \text{subject to} \quad \sum_{i=1}^n p_i = 1$$
- What is the solution?

# Optimal Importance Sampling

- The optimal sampling probabilities are  $p_i \propto k_i = k(x_i, y)$

# Optimal Importance Sampling

- The optimal sampling probabilities are  $p_i \propto k_i = k(x_i, y)$
- Meaning: Each point in  $X$  should be sampled with probability proportional to its contribution to the KDE

# Optimal Importance Sampling

- The optimal sampling probabilities are  $p_i \propto k_i = k(x_i, y)$
- Meaning: Each point in  $X$  should be sampled with probability proportional to its contribution to the KDE
- See a big problem here?

# Optimal Importance Sampling

- The optimal sampling probabilities are  $p_i \propto k_i = k(x_i, y)$
- Meaning: Each point in  $X$  should be sampled with probability proportional to its contribution to the KDE
- See a big problem here?
- Computing the optimal sampling probabilities is as slow as computing exact KDE
  - Not useful algorithmically

# Optimal Importance Sampling

- The optimal sampling probabilities are  $p_i \propto k_i = k(x_i, y)$
- Meaning: Each point in  $X$  should be sampled with probability proportional to its contribution to the KDE
- See a big problem here?
- Computing the optimal sampling probabilities is as slow as computing exact KDE
  - Not useful algorithmically
- Can we compute approximate importance sampling probabilities?
  - We need an efficient approximate way to estimate  $k(x_i, y)$
  - Do you know any?

# Approximate Importance Sampling for KDE

- We need an efficient way to sample  $x_i$  from  $X$  with probability  $p_i$  “related to”  $k(x_i, y)$ 
  - As long as  $p_i$  gets lower variance than uniform sampling, we’re happy

# Approximate Importance Sampling for KDE

- We need an efficient way to sample  $x_i$  from  $X$  with probability  $p_i$  “related to”  $k(x_i, y)$ 
  - As long as  $p_i$  gets lower variance than uniform sampling, we’re happy
- **Note:** we cannot compute the  $p_i$ ’s ahead of time! **Why?**

# Approximate Importance Sampling for KDE

- We need an efficient way to sample  $x_i$  from  $X$  with probability  $p_i$  “related to”  $k(x_i, y)$ 
  - As long as  $p_i$  gets lower variance than uniform sampling, we’re happy
- **Note:** we cannot compute the  $p_i$ ’s ahead of time! **Why?**
  - We want  $p_i$ ’s that lead to a smaller sample size, i.e., let us “touch” less points
  - But if we compute the  $p_i$ ’s in advance, we’ve already selected points to “touch” before ever starting importance sampling

# Approximate Importance Sampling for KDE

- We need an efficient way to sample  $x_i$  from  $X$  with probability  $p_i$  “related to”  $k(x_i, y)$ 
  - As long as  $p_i$  gets lower variance than uniform sampling, we’re happy
- **Note:** we cannot compute the  $p_i$ ’s ahead of time! **Why?**
  - We want  $p_i$ ’s that lead to a smaller sample size, i.e., let us “touch” less points
  - But if we compute the  $p_i$ ’s in advance, we’ve already selected points to “touch” before ever starting importance sampling
- **Thus:** We need a sampling process that **intrinsically** samples with good  $p_i$ ’s
- We will see how to do this with LSH

Questions?

# Final Project Information

- Reminder:
  - Choose either **deep reading** or **research** project
  - **Deep reading:**
    - Choose 1-3 papers related to class
    - Write a project that adds something over them
      - E.g., better or different presentation; unified exposition of a few papers; etc
  - **Research:** Try to make progress on an open problem related to class
  - Can fall back from research to deep reading at any time
- Grade will be based on **written final report** and on **presentation to instructor**

# Final Project Information: Timeline

- **By July 11:** Submit your [project proposal](#)
  - Chosen paper(s) and a short description of what you intend to do
  - Choose papers from lists on Moodle or other ones with my approval
  - You will get a yes/no answer for the proposal
  - You may submit your proposal at any time during the semester

# Final Project Information: Timeline

- **By July 11:** Submit your [project proposal](#)
  - Chosen paper(s) and a short description of what you intend to do
  - Choose papers from lists on Moodle or other ones with my approval
  - You will get a yes/no answer for the proposal
  - You may submit your proposal at any time during the semester
- **By August 1:** Submit your [project report](#)
  - Submit on Moodle only (no email submissions)
  - Details format and guidelines will be published later

# Final Project Information: Timeline

- **By July 11:** Submit your [project proposal](#)
  - Chosen paper(s) and a short description of what you intend to do
  - Choose papers from lists on Moodle or other ones with my approval
  - You will get a yes/no answer for the proposal
  - You may submit your proposal at any time during the semester
- **By August 1:** Submit your [project report](#)
  - Submit on Moodle only (no email submissions)
  - Details format and guidelines will be published later
- **~ Aug 9 –31:** Do your [project presentation](#) after I have had time to read them
  - I will publish time slots
  - Will likely be over zoom
  - If you need special scheduling accommodations, **work with me in advance**

# Final Project Information: Final Report

- 70% of final project grade

# Final Project Information: Final Report

- 70% of final project grade
- Adhere to [strict academic writing standards](#)
  - Language and presentation, scientific rigor, formatting, bibliography, ...

# Final Project Information: Final Report

- 70% of final project grade
- Adhere to [strict academic writing standards](#)
  - Language and presentation, scientific rigor, formatting, bibliography, ...
- Dive into [mathematical proofs and details](#)
  - No expository “high level” reports lacking mathematical depth

# Final Project Information: Final Report

- 70% of final project grade
- Adhere to **strict academic writing standards**
  - Language and presentation, scientific rigor, formatting, bibliography, ...
- Dive into **mathematical proofs and details**
  - No expository “high level” reports lacking mathematical depth
- Choose or formulate **key result to focus on** (the report will have a page limit)
  - Goal: Present a result that conveys the **key algorithmic and analytic ideas** of the paper – “separate the wheat from the chaff”
  - **Strike a balance**: don’t over-simplify; don’t get mired in unnecessary details
  - Part of the challenge you are evaluated on is **distilling these main idea**

# Final Project Information: Final Presentation

- 30% of final project grade

# Final Project Information: Final Presentation

- 30% of final project grade
- Will consist of two parts:
  - Giving a presentation you will have prepared
  - Answering questions about your final report

# Final Project Information: Final Presentation

- 30% of final project grade
- Will consist of two parts:
  - Giving a presentation you will have prepared
  - Answering questions about your final report
- Final presentations too are expected to **dive into mathematical details and proofs**
  - Make sure you fully refresh your memory on the report you submitted

# Final Project Information: Final Presentation

- 30% of final project grade
- Will consist of two parts:
  - Giving a presentation you will have prepared
  - Answering questions about your final report
- Final presentations too are expected to **dive into mathematical details and proofs**
  - Make sure you fully refresh your memory on the report you submitted
- Evaluated on quality of presentation (content, clarity, etc.) and on demonstrated understanding and command of the report and the papers it is based on

# Final Project Information: Final Presentation

- 30% of final project grade
- Will consist of two parts:
  - Giving a presentation you will have prepared
  - Answering questions about your final report
- Final presentations too are expected to **dive into mathematical details and proofs**
  - Make sure you fully refresh your memory on the report you submitted
- Evaluated on quality of presentation (content, clarity, etc.) and on demonstrated understanding and command of the report and the papers it is based on
- **Questions?**